

MODULE 18 · DOMAIN 4 - DATA SECURITY AND GOVERNANCE

Authorisation

IAM policies, AWS Lake Formation, column and row level control, and granting access across accounts.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 18 of 27

IAM works on **resources**. A column is not a resource.

Where permissions can be expressed

LAYER	GRANULARITY
IAM identity policy	Actions on resources. Buckets, prefixes, tables, jobs
Resource policy	Who may use this bucket, queue, key or topic
Lake Formation	Databases, tables, columns, rows and cells in the catalog
Redshift GRANT	Schemas, tables, columns and rows inside the warehouse
BI tool row and column security	What a dashboard viewer sees

ON THE EXAM

Each layer sees a different unit. Choosing the wrong layer for a requirement is the most common error here.

What a policy statement contains

ELEMENT	WHAT IT SAYS
Effect	Allow or Deny
Action	Which API operations, such as s3:GetObject
Resource	Which ARNs the actions apply to
Principal	Who, in a resource-based policy only
Condition	When it applies. Source VPC, tag, encryption, time

ON THE EXAM

Condition is where a broad permission becomes a narrow one. It is the element people use least and gain most from.

Conditions that matter for data

- ◆ **aws:SourceVpce** requires the request to arrive through a named VPC endpoint.
- ◆ **aws:PrincipalOrgID** restricts access to identities inside your organisation.
- ◆ **s3:prefix** limits which part of a bucket a listing or read may touch.
- ◆ **Encryption conditions** refuse an upload that does not specify the required key.
- ◆ **Tag conditions** are how attribute-based access control works in IAM.

ON THE EXAM

A condition requiring a specific KMS key on upload is how you guarantee everything in a bucket is encrypted correctly.

Managed or inline policies

- ◆ **AWS managed policies** are maintained by AWS and often broader than you need.
- ◆ **Customer managed policies** are yours, reusable across roles, and versioned.
- ◆ **Inline policies** attach to one identity and disappear with it.
- ◆ **Prefer customer managed** for anything more than one role uses.
- ◆ **An AWS managed policy granting full access** is convenient and rarely least privilege.

ON THE EXAM

AWS managed policies are a starting point, not a destination. Full access variants are the common shortcut in a rush.

Delegating without giving everything away

- ◆ A **permissions boundary** caps what a role can ever be granted.
- ◆ Let teams create their own roles, within limits they cannot exceed.
- ◆ Without one, anyone who can create roles can create an administrator.
- ◆ The boundary and the policy must both allow an action for it to work.
- ◆ Different from an SCP, which caps an account rather than a role.

ON THE EXAM

Permission to create roles is permission to escalate, unless a boundary caps what those roles may hold.

Lake Formation grants permissions on **catalog objects**, and the engines enforce them.

What Lake Formation can express

- ◆ **Table level.** This principal may select from this table.
- ◆ **Column level.** Include or exclude named columns from what they see.
- ◆ **Row level,** through a data filter with a filter expression.
- ◆ **Cell level,** combining a column selection with a row filter.
- ◆ **Enforced in Athena, Redshift Spectrum, EMR and Glue** from one grant.
- ◆ **Open table formats too,** including Iceberg, Hudi and Delta Lake.

ON THE EXAM

Data filters are the mechanism for row and cell level. They define both the columns and the row expression.

Lake Formation permission types

PERMISSION	WHAT IT ALLOWS
SELECT	Read the data. The permission most grants are about
DESCRIBE	See that the resource exists, without reading it
ALTER, DROP	Change or remove the table definition
INSERT, DELETE	Write or remove data through the catalog
CREATE_TABLE	Create tables within a database
DATA_LOCATION_ACCESS	Write to a registered Amazon S3 location

ON THE EXAM

Grantable permissions let a principal pass a permission on. Useful for delegation, and worth granting deliberately.

Who administers the lake

- ◆ **A data lake administrator** can grant permissions on any catalog resource.
- ◆ **Data owners grant on their own datasets**, using grantable permissions.
- ◆ **LF-Tag creators** are delegated the ability to define and assign tags.
- ◆ **An administrator with no restrictions** is a single point of failure and audit.
- ◆ **Separate defining tags from granting on them**, so one person cannot do both.

ON THE EXAM

Delegation is the point. A central team cannot approve every request in a lake with hundreds of datasets.

Named resources or LF-Tags?

NAMED RESOURCE GRANTS

Explicit

- Grant this principal on this table
- Obvious to read for a small estate
- One grant per principal per resource
- Grows unmanageably as tables multiply

TAG-BASED ACCESS CONTROL

By attribute

- Tag resources, grant on tag values
- New tables inherit access automatically
- Scales to thousands of tables
- Requires a tag ontology people agree on

THE TIEBREAKER

Three principals across three databases and seven tables is seventeen named grants, or a handful of tag grants.

How LF-Tags actually work

- ◆ **Key and value pairs**, such as classification equals restricted.
- ◆ **Not the same as IAM tags or S3 object tags**. A separate system entirely.
- ◆ **Attach to databases, tables or columns**, and they inherit downward.
- ◆ **Only one value per key per resource**. A table cannot be both sales and marketing.
- ◆ **Granting several tags at once** requires the resource to carry all of them.

ON THE EXAM

LF-Tags are Lake Formation's own tags. Confusing them with IAM tags or S3 object tags is a common error.

A hospital stores patient records in Amazon S3, catalogued in AWS Glue and queried with Amazon Athena. A research team must analyse the clinical columns but must never see patient name, address or national identifier.

A data engineer proposes writing an IAM policy for the research team's role that denies access to those three columns.

Why will this not work, and what should be done instead?

- ☐ A IAM cannot express column-level permissions. Use Lake Formation column-level grants or a data filter
- ☐ B The IAM policy will work, but it must be attached to the bucket rather than the role
- ☐ C Create a second copy of the table without the sensitive columns and grant access to that
- ☐ D Encrypt the three columns with a KMS key the research team cannot use

◆ PAUSE AND DECIDE

Why will this not work, and what should be done instead?

- A** IAM cannot do columns. Use Lake Formation
IAM grants actions on resources named by ARN. A column has no ARN, so there is nothing for a policy to deny.
- B** Attach it to the bucket instead
A bucket policy has the same vocabulary problem. Objects and prefixes, not columns.
- C** A second redacted copy
It works, and creates two datasets to keep in step, doubling storage and creating a consistency problem.
- D** Encrypt the columns separately
Client-side column encryption is possible and complex. Lake Formation solves this directly.

THE SIGNAL

The premise of the proposal is impossible. Recognising that is faster than evaluating the other options.

Turning Lake Formation on where IAM already works

- ◆ **An existing lake has IAM permissions** that already work and must not break.
- ◆ **Hybrid access mode** lets Lake Formation govern some principals while others use IAM.
- ◆ **Migrate table by table**, rather than switching everything at once.
- ◆ **IAMAllowedPrincipals** is the legacy setting meaning everyone falls back to IAM.
- ◆ **Removing it** is what makes Lake Formation permissions actually take effect.

ON THE EXAM

A Lake Formation grant that appears to do nothing is usually IAMAllowedPrincipals still granting access underneath.

Registering the data location

- ◆ Register the S3 location with Lake Formation before permissions can apply to it.
- ◆ Lake Formation then vends temporary credentials scoped to what the principal may see.
- ◆ A principal with direct S3 permissions bypasses all of it, as module two showed.
- ◆ Remove direct S3 grants from consumers, or the fine-grained layer is decorative.
- ◆ Glue ETL roles need the location too, for jobs that write.

ON THE EXAM

Fine-grained access only holds if the only path to the data goes through the catalog. Leave a direct path and it is bypassed.

A company has implemented Lake Formation column-level grants excluding three sensitive columns from an analytics team. Testing shows the team's Athena queries correctly return only the permitted columns.

A week later, an auditor discovers that the same team's AWS Glue job reads the underlying Parquet files directly from Amazon S3 and processes every column, including the three that were excluded.

What is the cause, and what should be done?

- | | |
|---|--|
| <p>A Lake Formation does not apply to Glue jobs. Nothing can be done</p> | <p>B The Glue role retains a direct S3 read policy, bypassing the catalog. Remove it and grant through Lake Formation</p> |
| <p>C Column-level grants only work in Athena. Use a redacted copy for Glue</p> | <p>D The three columns need to be encrypted at rest with a separate KMS key</p> |

◆ PAUSE AND DECIDE

What is the cause, and what should be done?

- A

 Lake Formation does not cover Glue
False. Lake Formation permissions are enforced in Glue ETL as well as Athena, Redshift Spectrum and EMR.
- B

 Direct S3 access bypasses the catalog
Enforcement happens when an engine consults the catalog. A role reading files directly never asks, so the grants never apply.
- C

 Athena only, use a redacted copy
The premise is wrong, and a second copy reintroduces the duplication problem scenario one rejected.
- D

 Encrypt the columns
Addresses storage encryption. The job would still read what its role permits, and the role permits everything.

THE SIGNAL

Athena worked and Glue did not. Same grants, two paths to the data, and only one goes through the catalog.

A limitation worth knowing

- ◆ **Glue ETL jobs require full table access** to run successfully.
- ◆ **Column-level LF-Tags can therefore break a Glue job** that needs every column.
- ◆ **Data filters are not affected** by this limitation, so use them for column filtering.
- ◆ **Give ETL roles full access** and restrict the consumers instead.
- ◆ **Restrict at the consumption layer**, which is where the requirement usually is anyway.

ON THE EXAM

Applying column-level LF-Tags can leave a Glue role with Describe only. Data filters avoid this, and so does role separation.

A view, or a permission?

A VIEW

A second object

- Works in any SQL engine
- Restricts columns and rows by definition
- One object per audience to maintain
- Consumers must be told to use it

A PERMISSION

The same object

- One table, filtered per principal
- Nothing extra to keep in step
- Consumers write ordinary queries
- Needs Lake Formation or Redshift RLS

THE TIEBREAKER

A view relies on nobody querying the base table. A permission holds however the data is reached.

Data filters in detail

- ◆ A **named filter** on a table, defining columns and a row expression.
- ◆ **Include or exclude columns**, and exclude is safer as tables gain columns.
- ◆ A **row filter expression** is SQL-like, such as region equals Europe.
- ◆ **All rows** is the expression when only column filtering is wanted.
- ◆ **Grant the filter to a principal**, rather than granting the table directly.

ON THE EXAM

Exclude columns rather than include them. A new sensitive column added next year is then hidden by default.

Sharing across accounts

- ◆ **Lake Formation shares catalog resources** through AWS Resource Access Manager.
- ◆ **Column and row filters survive the share**, so a consumer never sees excluded data.
- ◆ **Cross-account version 3** enabled sharing directly to principals and to Organizations.
- ◆ **Version 4** added hybrid access mode across accounts.
- ◆ **Version 5, February 2026**, shares by wildcard, removing per-resource RAM limits.

ON THE EXAM

Filters travelling with the share is the point. You can share a table while excluding the columns you must not share.

Redshift has its own permission system,
and it is **not** Lake Formation.

Redshift permissions

- ◆ **GRANT and REVOKE** on schemas, tables, columns and procedures.
- ◆ **Roles** group privileges, and roles can be granted to roles.
- ◆ **Row-level security policies** attach to a table and filter by the session's identity.
- ◆ **Dynamic data masking** transforms column values by role at query time.
- ◆ **Datashares can be governed by Lake Formation**, adding fine-grained control to sharing.

ON THE EXAM

Dynamic data masking means one table serves both audiences. No second copy, no separate view to maintain.

Fine-grained access in DynamoDB

- ◆ **IAM conditions on leading keys** restrict which items a principal may read.
- ◆ **dynamodb:LeadingKeys** ties access to the partition key value.
- ◆ **A user reads only their own items**, without a separate table each.
- ◆ **Attribute conditions** restrict which attributes are returned.
- ◆ This is IAM doing something close to row-level security, because the key is part of the request.

ON THE EXAM

DynamoDB is the exception to this module's opening rule, because the partition key appears in the request itself.

A retailer's Redshift warehouse holds a transactions table used by twelve country teams. Each team must see only their own country's rows. A support team must see all rows but with the customer email masked, while the fraud team must see it in full.

There are currently twelve views, one per country, plus two more for the support and fraud cases. Keeping them in step with schema changes has become a burden.

What should the data engineer implement?

- ☐ A Row-level security policies for the country filtering, and dynamic data masking on the email column by role
- ☐ B Keep the views but generate them automatically from a script when the schema changes
- ☐ C Move the table to Amazon S3 and use Lake Formation data filters
- ☐ D Create twelve separate tables, one per country, with the email column omitted where required

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 Row-level security plus dynamic data masking
One table, one schema. Rows filter by the session's role and the email column masks by role, with nothing to keep in step.
- B

 Generate the views automatically
Automates the maintenance of fourteen objects rather than removing them. The duplication remains.
- C

 Move to S3 and use Lake Formation
Relocates a warehouse table to the lake to reach a capability Redshift already has natively.
- D

 Twelve tables
Twelve copies of a schema, twelve pipelines to load, and the fraud and support requirements still unmet.

THE SIGNAL

Fourteen objects maintaining one table's worth of data. Both requirements have a native feature that removes them.

Permissions on the services themselves

- ◆ **Who may start a Glue job**, as distinct from what the job may read.
- ◆ **Who may run an Athena query**, and in which workgroup.
- ◆ **Who may invoke a Lambda function** or start a Step Functions execution.
- ◆ **Separate the operator from the pipeline.** A person triggering a job is not the job.
- ◆ **Granting a person the job's role** gives them everything the pipeline can do.

ON THE EXAM

The last point is the common mistake. An engineer needing to rerun a job does not need the job's data permissions.

Authorisation, service by service

SERVICE	WHERE PERMISSIONS LIVE
AWS Glue	IAM for job and catalog operations; Lake Formation for data
Amazon Athena	IAM for queries and workgroups; Lake Formation for tables
Amazon EMR	IAM for the cluster; runtime roles for per-job data access
Amazon MWAA	IAM for the environment; the execution role for what DAGs may do
Amazon Redshift	IAM for the cluster; GRANT for data inside it
AWS Lambda	The execution role for what it may touch; a resource policy for who may invoke

ON THE EXAM

Almost every row splits the same way: one permission to operate the service, another for the data it reaches.

Shared clusters, different permissions

- ◆ A **shared EMR cluster** traditionally gives every job the cluster's role.
- ◆ **Runtime roles** let each job submission carry its own identity.
- ◆ **Lake Formation then applies** per job, so column grants work on a shared cluster.
- ◆ **EMR on EKS** uses roles for service accounts, one per pod.
- ◆ **MWAA** has one execution role for the environment, so DAGs share permissions.

ON THE EXAM

The last row is a real design constraint. Separating sensitive workflows means separating MWAA environments.

Controlling who queries what

- ◆ **Athena workgroups** separate teams, with their own result location and limits.
- ◆ **IAM permission on the workgroup** decides who may run queries in it.
- ◆ **Query results are objects in S3**, and need their own access control.
- ◆ **A result file can contain the data a column grant excluded** if it was written by someone permitted.
- ◆ **Encrypt results**, and restrict the results location per workgroup.

ON THE EXAM

Fine-grained access on a table does not extend to the result files a permitted user produces from it.

Role-based and attribute-based access

- ◆ **Role-based** assigns permissions to a role, and people to roles.
- ◆ **Scales with people**, because a new joiner gets an existing role.
- ◆ **Does not scale with resources**. Each new table needs adding to each role.
- ◆ **Attribute-based** matches tags on the principal to tags on the resource.
- ◆ **Scales with both**, which is why LF-TBAC exists for the catalog.

ON THE EXAM

Skill 4.2.6 names role-based access control and least privilege. Attribute-based is how it survives growth.

Granting to groups, not people

- ◆ **Granting to individuals** means auditing a list that changes every week.
- ◆ **Grant to a role or a group**, and manage membership separately.
- ◆ **Lake Formation accepts IAM Identity Center groups** as principals directly.
- ◆ **Membership then comes from the directory**, as module seventeen argued.
- ◆ **A leaver loses access** without anyone touching a data permission.

ON THE EXAM

Grants should name roles and groups. Naming individuals is how a permission outlives the person it was for.

A data platform team manages a lake with about nine hundred tables across four domains, growing by roughly thirty tables a month. Access is granted with named-resource Lake Formation grants, and the team spends significant time processing access requests and adding new tables to existing grants.

New tables are frequently missed, so analysts discover they cannot see data that should have been available to them.

What should the team implement?

- ☐ A Grant each team access to all tables in their domain's database, at database level
- ☐ B Adopt LF-Tags, tagging tables by domain and sensitivity, and granting principals against tag values
- ☐ C Write a Lambda function that runs nightly and adds new tables to the existing grants
- ☐ D Move to IAM policies with wildcards on the Glue Data Catalog

◆ PAUSE AND DECIDE

What should the team implement?

- A

 Database-level grants
Removes the per-table work and the ability to restrict sensitive tables within a domain. Too blunt for a lake with mixed sensitivity.
- B

 LF-Tags by domain and sensitivity
A new table tagged on creation is covered automatically, and sensitivity remains expressible independently of domain.
- C

 A nightly job adding new tables
Automates a symptom of the wrong model, and leaves up to a day where new data is invisible.
- D

 IAM wildcards on the catalog
Returns to the layer that cannot express column-level control, which is what Lake Formation was adopted for.

THE SIGNAL

Thirty new tables a month with named grants is the exact arithmetic LF-Tags exist to remove.

Which layer for which requirement

REQUIREMENT	LAYER
Hide columns from a team, in the lake	Lake Formation column grants or a data filter
Show only a region's rows, in the lake	A Lake Formation data filter
Hide columns in the warehouse	Redshift GRANT on columns, or dynamic data masking
Show only a region's rows in the warehouse	Redshift row-level security
Restrict a whole bucket or prefix	An IAM or bucket policy
Restrict what a dashboard viewer sees	Row and column security in the BI tool

ON THE EXAM

The same requirement has a different answer depending on where the data sits. That is the distinction to hold.

Answering who can see this

- ◆ **Lake Formation lists grants** per resource and per principal.
- ◆ **CloudTrail records the grant itself**, so changes have an author and a time.
- ◆ **Data access is also logged**, so who actually read a table is answerable.
- ◆ **Effective access is harder** than listed grants, because tags and inheritance combine.
- ◆ **Test with a real principal** rather than reasoning about the policy.

ON THE EXAM

An auditor asks who can see this table. Listing grants answers it only if you also resolve tags and inheritance.

Keeping permissions honest over time

- ◆ **Access Analyzer unused findings** show roles and permissions nobody exercises.
- ◆ **Lake Formation records grants**, so who can see what is answerable.
- ◆ **Review on a schedule**, because permissions only accumulate.
- ◆ **Time-bound access** for temporary needs, rather than a permanent grant.
- ◆ **A grant made for a project three years ago** is still in force today.

ON THE EXAM

Nothing removes a permission automatically. Every access estate grows unless somebody prunes it deliberately.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Lake Formation grants have no effect	IAMAllowedPrincipals still present on the resource
Athena respects column grants, Glue does not	The Glue role has direct S3 access, bypassing the catalog
A Glue job loses write access after tagging	Column-level LF-Tags leave only Describe. Use data filters
Cross-account share hits a limit	RAM association limits. Cross-account version 5 shares by wildcard
Access requests take days	Named-resource grants at a scale that needs LF-Tags
A team sees a table nobody granted them	A database-level or tag grant broader than intended

ON THE EXAM

The first two rows are the classic Lake Formation surprises. Both are about a path that bypasses enforcement.

Reading an authorisation question quickly

- ◆ **Columns or rows?** IAM cannot do either. That eliminates options immediately.
- ◆ **Lake or warehouse?** Lake Formation for one, Redshift GRANT for the other.
- ◆ **How many tables?** Hundreds and growing means LF-Tags, not named grants.
- ◆ **Is there another path to the data?** Direct S3 access defeats everything.
- ◆ **Operating or reading?** Starting a job and reading its data are different grants.

ON THE EXAM

Next: encryption, masking and protecting personal data.